

深圳大学实验报告

课程名称：人工智能

实验项目名称：基于 MobileNetV2 的垃圾图像分类

学 院：电子与信息工程学院

专 业：电子信息工程

指导教师：戴林蕙

报告人：侯泓宇 学号：2024280180 班级：电子信息工程 01 班

实验时间：2026 年 5 月 1 日

实验报告提交时间：2026 年 5 月 10 日

教 务 部 制

一、实验目的与要求：

- 理解图像分类任务与目标检测任务的区别，掌握分类网络的基本原理。
- 了解迁移学习（Transfer Learning）的概念：利用在大规模数据集上预训练好的模型，在小数据集上快速微调，以较少的训练时间获得较好的分类效果。
- 掌握使用 PyTorch 框架加载 MobileNetV2 预训练模型并替换分类头的方法。
- 能够在无 GPU 的普通电脑上完成垃圾图像分类模型的训练与预测，理解训练过程中损失值与准确率的变化规律。
- 学会对预测结果进行可视化分析，判断模型在各类别上的表现差异。

二、实验方法与步骤：

1. 激活已有虚拟环境并安装依赖

直接复用上次实验创建的 yolo11 虚拟环境，打开 Anaconda Prompt：

```
1 conda activate yolo11
2 pip install torch torchvision matplotlib
```

2. 准备数据集

本实验使用已准备好的垃圾分类数据集，包含 6 个类别：纸板（cardboard）、玻璃（glass）、金属（metal）、纸张（paper）、塑料（plastic）、其他垃圾（trash）。

将数据集文件夹与脚本放在同一目录下，结构如下：

```
1 项目文件夹/
2   train.py
3   predict.py
4   garbage-classification/
5       cardboard/      # 纸板图片
6       glass/          # 玻璃图片
7       metal/          # 金属图片
8       paper/          # 纸张图片
9       plastic/        # 塑料图片
10      trash/          # 其他垃圾图片
```

3. 编写并运行训练脚本

新建 `train.py` (内容见第三节), 然后运行:

```
1 python train.py
```

注意: 无 GPU 的电脑训练 15 个 epoch 约需 10-20 分钟, 训练完成后权重自动保存为 `best_model.pth`。

4. 编写并运行预测脚本

准备一张测试图片 (如手机拍摄的垃圾照片), 重命名为 `test.jpg` 放入项目文件夹, 然后运行:

```
1 python predict.py
```

终端会输出预测类别与置信度, 同时弹出带标注的图片窗口。

(1) 训练脚本 train.py

[illegible]

```

33 val_data    = datasets.ImageFolder('garbage-classification',
34                                     transform=val_transform)
35 class_names = train_data.classes
36 print("类别: ", class_names)
37
38 # 按相同的随机种子划分索引, 保证两个 dataset 划分一致
39 n_total = len(train_data)
40 n_train = int(n_total * 0.8)
41 n_val    = n_total - n_train
42 indices = torch.randperm(n_total).tolist()
43 train_indices = indices[:n_train]
44 val_indices   = indices[n_train:]
45
46 from torch.utils.data import Subset
47 train_set = Subset(train_data, train_indices)
48 val_set   = Subset(val_data,   val_indices)
49
50 train_loader = DataLoader(train_set, batch_size=16, shuffle=True)
51 val_loader   = DataLoader(val_set,   batch_size=16, shuffle=False)
52 print(f"训练集: {n_train} 张, 验证集: {n_val} 张")
53
54 # 3. 加载预训练模型并替换分类头
55 # MobileNetV2 是轻量级网络, 适合 CPU 运行
56 model = models.mobilenet_v2(weights='IMAGENET1K_V1')
57
58 # 冻结主干网络, 只训练最后的分类头 (加快训练速度)
59 for param in model.features.parameters():
60     param.requires_grad = False
61
62 # 将原来的 1000 类分类头替换为 6 类
63 model.classifier[1] = nn.Linear(model.last_channel, len(class_
    names))
64 print("模型加载完成, 开始训练...")
65
66 # 4. 训练配置
67 device    = torch.device('cpu')          # 无 GPU 使用 CPU
68 model     = model.to(device)

```

```

69 criterion = nn.CrossEntropyLoss()           # 多分类交叉熵损失
70 optimizer = torch.optim.Adam(
71     model.classifier.parameters(), lr=0.001)
72
73 # 5. 训练循环
74 EPOCHS = 15
75 best_acc = 0.0
76
77 for epoch in range(EPOCHS):
78     # 训练阶段
79     model.train()
80     total_loss, correct = 0, 0
81     for imgs, labels in train_loader:
82         imgs, labels = imgs.to(device), labels.to(device)
83         optimizer.zero_grad()
84         outputs = model(imgs)
85         loss = criterion(outputs, labels)
86         loss.backward()
87         optimizer.step()
88         total_loss += loss.item()
89         correct += (outputs.argmax(1) == labels).sum().item()
90
91     train_acc = correct / n_train
92
93     # 验证阶段
94     model.eval()
95     val_correct = 0
96     with torch.no_grad():
97         for imgs, labels in val_loader:
98             imgs, labels = imgs.to(device), labels.to(device)
99             outputs = model(imgs)
100             val_correct += (outputs.argmax(1) == labels).sum().
                item()
101
102     val_acc = val_correct / n_val
103     print(f"Epoch_{epoch+1}/{EPOCHS} ")
104     f"Loss:_{total_loss/len(train_loader):.3f}"

```

```

105         f"Train_Acc:_{train\_acc:.3f}%"
106         f"Val_Acc:_{val\_acc:.3f}")
107
108     # 保存验证集最优权重
109     if val_acc > best_acc:
110         best_acc = val_acc
111         torch.save(model.state_dict(), 'best_model.pth')
112         print(f"%s已保存最优权重 (Val_Acc:{best\_acc:.3f}) ")
113
114 print(f"\n训练完成！ 最优验证准确率: {best_acc:.3f}")
115 print("权重已保存至best_model.pth")

```

(2) 预测脚本 predict.py

```

1  import torch
2  import torch.nn as nn
3  from torchvision import transforms, models
4  from PIL import Image
5  import matplotlib.pyplot as plt
6  import matplotlib
7  matplotlib.rcParams['font.sans-serif'] = ['SimHei'] # 支持中文标题
8
9  # 1. 类别名称 (与训练时一致)
10 class_names = ['cardboard (纸板)', 'glass (玻璃)', 'metal (金属)',
11               'paper (纸张)', 'plastic (塑料)', 'trash (其他)']
12
13 # 2. 加载模型
14 model = models.mobilenet_v2(weights=None)
15 model.classifier[1] = nn.Linear(model.last_channel, len(class_names))
16 model.load_state_dict(torch.load('best_model.pth', map_location='cpu'))
17 model.eval()
18
19 # 3. 图片预处理 (与验证集一致)

```

```

20 transform = transforms.Compose([
21     transforms.Resize((128, 128)),
22     transforms.ToTensor(),
23     transforms.Normalize(
24         mean=[0.485, 0.456, 0.406],
25         std=[0.229, 0.224, 0.225]),
26 ])
27
28 # 4. 读取并预测
29 img_path = 'test.jpg' # 替换为你自己的测试图片路径
30 img = Image.open(img_path).convert('RGB')
31 tensor = transform(img).unsqueeze(0) # 增加 batch 维度
32
33 with torch.no_grad():
34     outputs = model(tensor)
35     probs = torch.softmax(outputs, dim=1)[0] # 转为概率
36     pred = probs.argmax().item()
37
38 print(f"预测结果: {class_names[pred]}")
39 print(f"置信度: {probs[pred]:.2%}")
40
41 # 5. 可视化
42 plt.figure(figsize=(5, 5))
43 plt.imshow(img)
44 plt.axis('off')
45 plt.title(f"预测: {class_names[pred]}\n置信度: {probs[pred]:.2%}",
46         fontsize=14)
47 plt.tight_layout()
48 plt.savefig('result.jpg') # 保存结果图
49 plt.show()

```

四、数据处理与分析:

以 CPU 训练 10 个 epoch 的典型结果为例:

Epoch	Loss	Train Acc	Val Acc
1	1.199	0.545	0.638
3	0.779	0.722	0.685
5	0.772	0.716	0.735
8	0.698	0.743	0.739
11	0.684	0.752	0.745

各指标含义：

- **Loss**：交叉熵损失，越低说明模型预测的概率分布越接近真实标签。
- **Train Acc**：训练集准确率，反映模型对训练数据的拟合程度。
- **Val Acc**：验证集准确率，反映模型对未见过数据的泛化能力，是评估模型好坏的主要指标。

从实验结果可以看出，模型在训练过程中整体表现稳定，Loss 由 1.199 逐步下降至 0.684，说明模型的预测结果与真实标签之间的差距不断减小；同时，训练集准确率和验证集准确率均呈上升趋势，最终验证集准确率达到 0.745，表明模型具备一定的分类能力和泛化能力。但从结果也可以看出，模型仍存在进一步优化空间，后续可通过增加训练轮数、调整学习率、优化网络结构或扩充数据集等方式提升模型性能。

五、实验结论：

1. 本实验在无 GPU 的普通电脑上，利用 MobileNetV2 预训练权重进行迁移学习，成功完成了垃圾图像六分类任务，验证集准确率达到约 74%，训练仅需 10-20 分钟，展示了迁移学习在低配硬件上的高效性。
2. 迁移学习的核心优势在于：冻结主干网络的特征提取层，只训练最后的分类头，大幅减少了需要更新的参数量，使得小数据集上也能快速收敛并获得较好效果。
3. MobileNetV2 相比 ResNet 等较大模型，参数量更少、推理速度更快，适合在 CPU 或移动端设备上部署，是实际工程中常用的轻量级分类网络。
4. 实验过程中发现，塑料和玻璃类别因外观特征相近，存在一定误分类现象。后续可通过增加该类别的训练样本或引入更强的数据增强策略加以改善。
5. 与前两次实验（目标检测、目标跟踪）相比，图像分类任务逻辑更为简洁，加深了对“输入图像 → 特征提取 → 分类输出”这一基本深度学习流程的理解。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：

无

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。